

```

1  """
2  Programming Assignment 1 – Q-Learning for Acrobot
3  =====
4
5  Install dependencies:
6      pip install gymnasium[classic-control] numpy matplotlib pandas
7
8  Run:
9      python q_learning_acrobot.py
10
11  Output artifacts:
12      results.csv          – per-episode training history
13      summary_metrics.csv – final metric table for comparisons
14      checkpoint_success.csv – periodic greedy-evaluation success rates
15      figures/             – four PNG plots
16      Console summary      – key metrics
17
18  Performance metrics tracked:
19      episode return, moving-average return, episode length, success flag,
20      checkpoint success rate, final evaluation mean/std reward, final
21      evaluation success rate, final evaluation mean episode length, training
22      time, and sample-efficiency threshold crossing.
23
24  Visualizations generated:
25      reward curve, episode-length curve, checkpoint success-rate curve,
26      and epsilon-decay curve.
27  """
28
29  import os
30  import time
31
32  import gymnasium as gym
33  import matplotlib.pyplot as plt
34  import numpy as np
35  import pandas as pd
36
37  # -----
38  # CONFIG – all hyperparameters in one place
39  # -----
40  CONFIG = {
41      "seed": 42,
42      # Discretization
43      "state_low": np.array([-1.0, -1.0, -1.0, -1.0, -12.57, -28.27]),
44      "state_high": np.array([ 1.0,  1.0,  1.0,  1.0,  12.57,  28.27]),
45      "num_bins": np.array([8, 8, 8, 8, 12, 12]),
46      # Q-learning
47      "alpha": 0.1, # learning rate
48      "gamma": 0.99, # discount factor

```

```

49     "epsilon":      1.0,      # initial exploration rate
50     "epsilon_min":  0.05,
51     "epsilon_decay": 0.995,
52     # Training
53     "num_episodes": 5000,
54     "max_steps":    500,
55     "eval_interval": 500,      # evaluate every N training episodes
56     # Evaluation
57     "eval_episodes": 100,
58     # Plotting
59     "ma_window":     100,      # moving-average window
60     # Sample-efficiency threshold: first episode where 100-ep MA exceeds this
61     "efficiency_threshold": -200.0,
62 }
63
64
65 # -----
66 # Phase 2 – Environment & discretization
67 # -----
68
69 def make_env(seed: int) -> gym.Env:
70     env = gym.make("Acrobot-v1")
71     env.reset(seed=seed)
72     env.action_space.seed(seed)
73     print(f"Observation space : {env.observation_space}")
74     print(f"Action space      : {env.action_space}")
75     return env
76
77
78 def discretize_state(state: np.ndarray, cfg: dict) -> tuple:
79     """Convert a continuous 6-D observation to a discrete tuple index."""
80     clipped = np.clip(state, cfg["state_low"], cfg["state_high"])
81     ratios = (clipped - cfg["state_low"]) / (cfg["state_high"] - cfg["state_low"])
82     indices = np.floor(ratios * cfg["num_bins"]).astype(int)
83     indices = np.clip(indices, 0, cfg["num_bins"] - 1)
84     return tuple(indices)
85
86
87 def _sanity_check_discretizer(env: gym.Env, cfg: dict) -> None:
88     obs, _ = env.reset(seed=cfg["seed"])
89     idx = discretize_state(obs, cfg)
90     assert isinstance(idx, tuple) and len(idx) == 6, "discretize_state must return a 6-tuple"
91     assert all(isinstance(i, (int, np.integer)) for i in idx), "all indices must be integers"
92
93
94 # -----
95 # Phase 3 – Q-table & policy
96 # -----

```

```

97
98 def init_q_table(cfg: dict, action_size: int) -> np.ndarray:
99     shape = tuple(cfg["num_bins"]) + (action_size,)
100     return np.zeros(shape)
101
102
103 def choose_action(rng: np.random.Generator, q_table: np.ndarray, state_idx: tuple,
104                 epsilon: float, action_size: int) -> int:
105     if rng.random() < epsilon:
106         return int(rng.integers(action_size))
107     return int(np.argmax(q_table[state_idx]))
108
109
110 def update_q(q_table: np.ndarray, state_idx: tuple, action: int,
111             reward: float, next_state_idx: tuple, done: bool,
112             alpha: float, gamma: float) -> None:
113     best_next = 0.0 if done else float(np.max(q_table[next_state_idx]))
114     td_target = reward + gamma * best_next
115     td_error = td_target - q_table[state_idx][action]
116     q_table[state_idx][action] += alpha * td_error
117
118
119 # -----
120 # Phase 5 – Evaluation (defined before training so it can be called inline)
121 # -----
122
123 def evaluate(env: gym.Env, q_table: np.ndarray, cfg: dict) -> dict:
124     """
125     Run `eval_episodes` greedy episodes.
126     Returns dict with keys: rewards, lengths, successes.
127     """
128     eval_episodes = cfg["eval_episodes"]
129     max_steps = cfg["max_steps"]
130     rewards, lengths, successes = [], [], []
131     for _ in range(eval_episodes):
132         obs, _ = env.reset()
133         state_idx = discretize_state(obs, cfg)
134         total_reward = 0.0
135         success = 0
136         for step in range(max_steps):
137             action = int(np.argmax(q_table[state_idx]))
138             obs, reward, terminated, truncated, _ = env.step(action)
139             done = terminated or truncated
140             state_idx = discretize_state(obs, cfg)
141             total_reward += reward
142             if terminated:
143                 success = 1
144             if done:

```

```

145         break
146         rewards.append(total_reward)
147         lengths.append(step + 1)
148         successes.append(success)
149     return {"rewards": rewards, "lengths": lengths, "successes": successes}
150
151
152 # -----
153 # Phase 4 – Training loop
154 # -----
155
156 def train(env: gym.Env, eval_env: gym.Env, q_table: np.ndarray,
157          cfg: dict, rng: np.random.Generator) -> tuple[np.ndarray, dict]:
158     """
159     Train the agent. Returns (updated q_table, history).
160     history keys: rewards, lengths, successes, epsilons, checkpoint_episodes,
161                  checkpoint_success_rates, training_time.
162     """
163     epsilon = cfg["epsilon"]
164     action_size = env.action_space.n
165     alpha = cfg["alpha"]
166     gamma = cfg["gamma"]
167     max_steps = cfg["max_steps"]
168     ma_window = cfg["ma_window"]
169     eval_interval = cfg["eval_interval"]
170     num_episodes = cfg["num_episodes"]
171     history = {
172         "rewards": [],
173         "lengths": [],
174         "successes": [],
175         "epsilons": [],
176         "checkpoint_episodes": [],
177         "checkpoint_success_rates": [],
178     }
179
180     t_start = time.time()
181
182     for episode in range(1, num_episodes + 1):
183         obs, _ = env.reset()
184         state_idx = discretize_state(obs, cfg)
185         total_reward = 0.0
186         success = 0
187         ep_length = max_steps
188
189         for step in range(max_steps):
190             action = choose_action(rng, q_table, state_idx, epsilon, action_size)
191             obs, reward, terminated, truncated, _ = env.step(action)
192             done = terminated or truncated

```

```

193         next_state_idx = discretize_state(obs, cfg)
194
195         update_q(q_table, state_idx, action, reward, next_state_idx, done,
196                 alpha, gamma)
197
198         state_idx = next_state_idx
199         total_reward += reward
200
201         if terminated:
202             success = 1
203         if done:
204             ep_length = step + 1
205             break
206
207         history["rewards"].append(total_reward)
208         history["lengths"].append(ep_length)
209         history["successes"].append(success)
210         history["epsilons"].append(epsilon)
211
212         # Decay epsilon
213         epsilon = max(cfg["epsilon_min"], epsilon * cfg["epsilon_decay"])
214
215         # Periodic evaluation checkpoint
216         if episode % eval_interval == 0:
217             ckpt = evaluate(eval_env, q_table, cfg)
218             success_rate = 100.0 * sum(ckpt["successes"]) / len(ckpt["successes"])
219             history["checkpoint_episodes"].append(episode)
220             history["checkpoint_success_rates"].append(success_rate)
221
222         # Progress print
223         recent_rewards = history["rewards"][-ma_window:]
224         ma = np.mean(recent_rewards)
225         print(
226             f"Episode {episode:5d}/{num_episodes} | "
227             f"MA Reward(100): {ma:8.1f} | "
228             f"Epsilon: {epsilon:.4f} | "
229             f"Eval success: {success_rate:.1f}%"
230         )
231
232         history["training_time"] = time.time() - t_start
233         return q_table, history
234
235
236 # -----
237 # Phase 6 – Metrics & CSV
238 # -----
239
240 def compute_metrics(history: dict, final_eval: dict, cfg: dict) -> dict:

```

```
241     rewards = np.array(history["rewards"])
242     w = cfg["ma_window"]
243
244     # Moving average (full series)
245     ma_rewards = np.convolve(rewards, np.ones(w) / w, mode="valid")
246     best_ma = float(np.max(ma_rewards)) if len(ma_rewards) > 0 else float("nan")
247
248     # Sample efficiency
249     threshold = cfg["efficiency_threshold"]
250     crossing = next(
251         (i + w for i, v in enumerate(ma_rewards) if v >= threshold),
252         None
253     )
254
255     eval_rewards = np.array(final_eval["rewards"])
256     eval_lengths = np.array(final_eval["lengths"])
257     eval_successes = np.array(final_eval["successes"])
258
259     return {
260         "total_episodes":      cfg["num_episodes"],
261         "final_epsilon":      round(history["epsilons"][-1], 5),
262         "best_ma_reward":      round(best_ma, 2),
263         "eval_mean_reward":    round(float(np.mean(eval_rewards)), 2),
264         "eval_std_reward":     round(float(np.std(eval_rewards)), 2),
265         "eval_success_rate_pct": round(100.0 * float(np.mean(eval_successes)), 1),
266         "eval_mean_ep_length": round(float(np.mean(eval_lengths)), 1),
267         "training_time_sec":   round(history["training_time"], 1),
268         "sample_efficiency_ep": crossing,
269     }
270
271
272 def save_csv(history: dict, path: str = "results.csv") -> None:
273     df = pd.DataFrame({
274         "episode": range(1, len(history["rewards"]) + 1),
275         "reward":   history["rewards"],
276         "length":   history["lengths"],
277         "success":  history["successes"],
278         "epsilon":  history["epsilons"],
279     })
280     df.to_csv(path, index=False)
281     print(f"Saved training history → {path}")
282
283
284 def save_summary_csv(metrics: dict, path: str = "summary_metrics.csv") -> None:
285     df = pd.DataFrame([metrics])
286     df.to_csv(path, index=False)
287     print(f"Saved summary metrics → {path}")
288
```

```

289
290 def save_checkpoint_csv(history: dict, path: str = "checkpoint_success.csv") -> None:
291     df = pd.DataFrame({
292         "episode": history["checkpoint_episodes"],
293         "success_rate_pct": history["checkpoint_success_rates"],
294     })
295     df.to_csv(path, index=False)
296     print(f"Saved checkpoint success rates → {path}")
297
298
299 def print_summary(metrics: dict) -> None:
300     print("\n" + "=" * 52)
301     print(" SUMMARY TABLE")
302     print("=" * 52)
303     rows = [
304         ("Total training episodes", metrics["total_episodes"]),
305         ("Final epsilon", metrics["final_epsilon"]),
306         ("Best 100-ep moving avg reward", metrics["best_ma_reward"]),
307         ("Eval mean reward", metrics["eval_mean_reward"]),
308         ("Eval std reward", metrics["eval_std_reward"]),
309         ("Eval success rate (%)", metrics["eval_success_rate_pct"]),
310         ("Eval mean episode length", metrics["eval_mean_ep_length"]),
311         ("Training time (s)", metrics["training_time_sec"]),
312         ("Sample efficiency (episode)",
313          metrics["sample_efficiency_ep"] if metrics["sample_efficiency_ep"] else "Not reached"),
314     ]
315     for label, value in rows:
316         print(f" {label:<34} {value}")
317     print("=" * 52 + "\n")
318
319
320 # -----
321 # Phase 7 – Visualizations
322 # -----
323
324 def _moving_average(data: list, window: int) -> np.ndarray:
325     return np.convolve(data, np.ones(window) / window, mode="valid")
326
327
328 def plot_reward_curve(history: dict, cfg: dict, out_dir: str) -> None:
329     rewards = history["rewards"]
330     ma = _moving_average(rewards, cfg["ma_window"])
331     episodes = range(1, len(rewards) + 1)
332     ma_episodes = range(cfg["ma_window"], len(rewards) + 1)
333
334     plt.figure(figsize=(10, 4))
335     plt.plot(episodes, rewards, alpha=0.25, color="steelblue", label="Episode reward")
336     plt.plot(ma_episodes, ma, color="steelblue", linewidth=2,

```

```
337         label=f"{cfg['ma_window']}-ep moving average")
338     plt.axhline(-500, color="red", linestyle="--", linewidth=0.8, label="Worst possible")
339     plt.xlabel("Episode")
340     plt.ylabel("Total reward")
341     plt.title("Q-Learning – Episode Reward (Acrobot-v1)")
342     plt.legend()
343     plt.tight_layout()
344     path = os.path.join(out_dir, "reward_curve.png")
345     plt.savefig(path, dpi=150)
346     plt.close()
347     print(f"Saved plot → {path}")
348
349
350 def plot_length_curve(history: dict, cfg: dict, out_dir: str) -> None:
351     lengths = history["lengths"]
352     ma = _moving_average(lengths, cfg["ma_window"])
353     episodes = range(1, len(lengths) + 1)
354     ma_episodes = range(cfg["ma_window"], len(lengths) + 1)
355
356     plt.figure(figsize=(10, 4))
357     plt.plot(episodes, lengths, alpha=0.25, color="darkorange", label="Episode length")
358     plt.plot(ma_episodes, ma, color="darkorange", linewidth=2,
359             label=f"{cfg['ma_window']}-ep moving average")
360     plt.xlabel("Episode")
361     plt.ylabel("Steps")
362     plt.title("Q-Learning – Episode Length (Acrobot-v1)")
363     plt.legend()
364     plt.tight_layout()
365     path = os.path.join(out_dir, "episode_length_curve.png")
366     plt.savefig(path, dpi=150)
367     plt.close()
368     print(f"Saved plot → {path}")
369
370
371 def plot_success_rate(history: dict, out_dir: str) -> None:
372     ckpt_eps = history["checkpoint_episodes"]
373     ckpt_rates = history["checkpoint_success_rates"]
374
375     plt.figure(figsize=(8, 4))
376     plt.plot(ckpt_eps, ckpt_rates, marker="o", color="seagreen", linewidth=2)
377     plt.xlabel("Training episode")
378     plt.ylabel("Success rate (%)")
379     plt.title("Q-Learning – Checkpoint Success Rate (Acrobot-v1)")
380     plt.ylim(0, 105)
381     plt.tight_layout()
382     path = os.path.join(out_dir, "success_rate.png")
383     plt.savefig(path, dpi=150)
384     plt.close()
```



```
385     print(f"Saved plot → {path}")
386
387
388 def plot_epsilon_decay(history: dict, out_dir: str) -> None:
389     plt.figure(figsize=(8, 3))
390     plt.plot(range(1, len(history["epsilons"]) + 1), history["epsilons"],
391             color="mediumpurple", linewidth=1.5)
392     plt.xlabel("Episode")
393     plt.ylabel("Epsilon")
394     plt.title("Q-Learning – Epsilon Decay (Acrobot-v1)")
395     plt.tight_layout()
396     path = os.path.join(out_dir, "epsilon_decay.png")
397     plt.savefig(path, dpi=150)
398     plt.close()
399     print(f"Saved plot → {path}")
400
401
402 # -----
403 # Entry point
404 # -----
405
406 def main() -> None:
407     rng = np.random.default_rng(CONFIG["seed"])
408     os.makedirs("figures", exist_ok=True)
409
410     print("\n=== Q-Learning for Acrobot-v1 ===\n")
411
412     # Phase 2
413     env = make_env(CONFIG["seed"])
414     eval_env = gym.make("Acrobot-v1")
415     eval_env.reset(seed=CONFIG["seed"] + 1)
416     _sanity_check_discretizer(env, CONFIG)
417
418     # Phase 3
419     q_table = init_q_table(CONFIG, env.action_space.n)
420     print(f"Q-table shape: {q_table.shape} "
421           f"({q_table.size:,} entries)\n")
422
423     # Phase 4 + 5 (training + periodic evaluation)
424     print("Training ...\n")
425     q_table, history = train(env, eval_env, q_table, CONFIG, rng)
426
427     # Phase 5 – final evaluation
428     print("\nRunning final evaluation ...")
429     final_eval = evaluate(eval_env, q_table, CONFIG)
430
431     # Phase 6
432     metrics = compute_metrics(history, final_eval, CONFIG)
```

```
433     save_csv(history)
434     save_summary_csv(metrics)
435     save_checkpoint_csv(history)
436     print_summary(metrics)
437
438     # Phase 7
439     print("Generating plots ...")
440     plot_reward_curve(history, CONFIG, "figures")
441     plot_length_curve(history, CONFIG, "figures")
442     plot_success_rate(history, "figures")
443     plot_epsilon_decay(history, "figures")
444
445     print("\nDone. See figures/ for plots, results.csv for training data, ")
446     print("summary_metrics.csv for final metrics, and checkpoint_success.csv for checkpoint results.")
447     env.close()
448     eval_env.close()
449
450
451 if __name__ == "__main__":
452     main()
```